

Mind Your Steps: A General Learning Framework for Accurate Humanoid Foothold Tracking

Alessandro Montenegro¹, Shihao Li², Puze Liu^{2,3,†}, Alberto Maria Metelli¹, and Jan Peters³⁻⁶

¹Politecnico di Milano ²Tongji University ³Technische Universität Darmstadt ⁴hessian.AI

⁵German Research Center for Artificial Intelligence (DFKI) ⁶Robotics Institute Germany (RIG)

Project Website: <https://montenegroalessandro.github.io/mind-your-steps/>

Abstract—Enabling humanoid robots to operate in complex, dynamic environments remains a critical challenge, fundamentally limited by the ability to navigate robustly, safely, and accurately. While reinforcement learning with velocity-commanded policies has achieved remarkable robustness in humanoid locomotion, this approach lacks explicit control of the foothold placement, leading to unsafe behavior, such as stepping onto human feet, or imprecise navigation, hindering the following manipulation task. Conversely, explicit foothold-tracking policies offer a promising alternative by directly being commanded with target foot poses. However, existing approaches are often limited by unrealistic state assumptions, compromising real-world deployment, or they are part of staged pipelines, making them tied to specific downstream tasks. In this work, we introduce a novel, lightweight framework for training general-purpose 3D foothold-tracking policies. By dynamically providing footstep support through a goal sampler, this method enables the learned policy to be agnostic to specific terrains. Our new target representation effectively mitigates challenges arising in the real world, such as noisy and inaccurate pose estimation and foot contact estimation. Designed for direct real-world transfer, our policy acts as a standalone low-level controller that can be seamlessly paired with various high-level foothold generators. We demonstrate the effectiveness of our framework through extensive experiments in simulation and in the real world. By coupling our policy with different upstream planners, we achieve natural and accurate locomotion in challenging settings, paving the way for loco-manipulation tasks in complex environments.

I. INTRODUCTION

Recent research on humanoid robotics has demonstrated remarkable progress, enabling humanoids to perform agile and highly dynamic behaviors such as backflips and flying kicks. Despite these achievements, enabling humanoids to operate autonomously in real-world environments remains a critical challenge. A fundamental prerequisite for real-world deployment is the ability to navigate safely, accurately, and reliably in everyday settings [19, 34, 3, 11]. In realistic environments, humanoid robots must contend with numerous challenges, including obstacle avoidance, interaction with humans, traversal of slopes and stairs, and locomotion over uneven or irregular terrain [50, 14, 42]. In such conditions, stable locomotion critically depends on precise and informed foot placement, as feasible footholds are often sparse and constrained. Consequently, endowing humanoids with con-

trollers capable of constraining foot placements to valid and safe contact regions explicitly is a critical problem.

Given its fundamental importance, legged locomotion has become one of the most studied fields in robotics. Extensive investigation, spanning model-based [34, 43, 16], data-driven [22, 20, 27, 23, 29, 51, 5, 12, 24, 3], and hybrid methodologies [48, 8, 45, 31, 17, 14], has yielded approaches with remarkable robustness in both simulation and real-world deployment. However, the predominant paradigm remains the integration of vision pipelines with velocity-commanded policies [42, 14, 11, 4]. In this setting, the control policy typically relies on height map observations and auxiliary reward terms to implicitly select valid footholds while steering the robot away from unsteppable areas. However, the inherent mismatch between continuous velocity commands and discrete foot placement results in degraded tracking accuracy and reduced efficiency in reaching the target.

Since the aforementioned methodologies do not allow for direct control over the robot’s foot placement, explicit foothold-commanded policies have recently emerged as a promising alternative paradigm [32, 44, 7, 14, 41]. These approaches are designed to enhance accuracy on foot placement targets and enable more flexible gait generation and motion capabilities. However, existing implementations remain severely limited in scope and practicality, far from general-purpose utility. Most have been studied solely in simulation [32, 44], exist only in preliminary forms restricted to simple forward walking on flat terrain [7], or are embedded as low-level components within specific, monolithic pipelines [32, 14]. Consequently, they often fail to support direct real-world usage and remain closely coupled to specific downstream tasks.

Original Contribution. Motivated by achieving stable and accurate locomotion via foothold-conditioned policies, in this work we propose a novel, lightweight, and model-free framework for training general-purpose 3D foothold-tracking policies. Such foothold-trackers are agnostic to the high-level planner and ready for immediate deployment, enabling seamless integration with diverse downstream tasks. Our specific contributions are as follows:

- We propose a framework designed to train policies relying solely on proprioception, gait phase, and stance-relative foothold targets. By expressing commands in the stance-foot frame, the goal remains constant throughout the

[†]Correspondence to: Puze Liu (puze_liu@tongji.edu.cn).

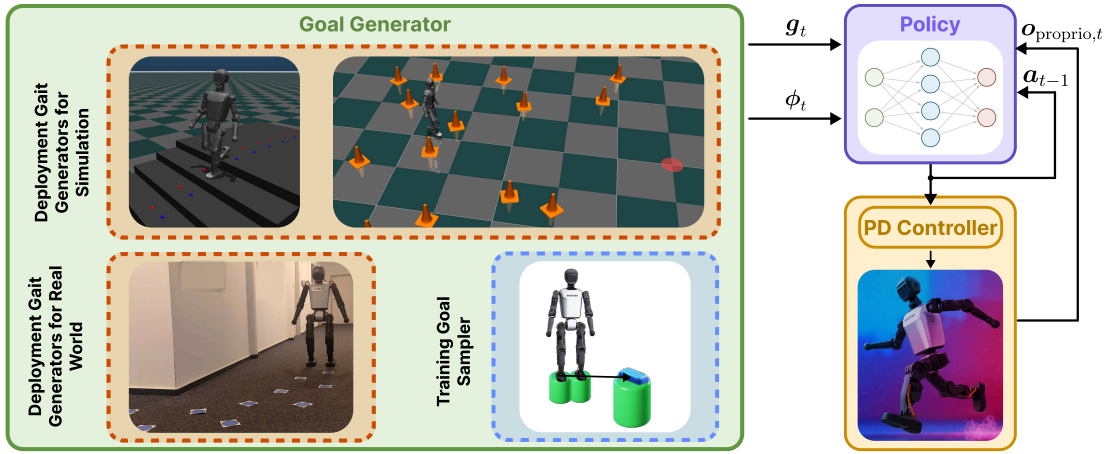


Fig. 1: **Training and Deployment Architecture.** Overview of the training and deployment architecture. *Policy.* The policy π_{θ} receives proprioception $o_{proprio,t}$, the previous action a_{t-1} , the foothold goal g_t , and the gait phase ϕ_t . It outputs joint position targets for an underlying PD controller. *Goal Generator.* This modular component supplies foothold targets to the policy. During *training*, a procedural Goal Sampler generates synthetic targets to enforce tracking accuracy. During *deployment* (simulation or real-world), this is replaced by task-specific planners, such as a vision-based estimator extracting visual targets from the ground or a teleoperation interface defining step direction, length, and yaw orientation.

swing phase, removing the need for precise base state estimation [32, 44]. To this end, we propose a novel Goal Sampler (GS) capable of generating feasible 3D foothold targets during training. Our GS does not tie with specific predefined terrains, enabling a general-purpose policy for downstream tasks. Coupled with specific reward terms balancing tracking accuracy with natural locomotion, this enables the agent to achieve accurate foot placement capabilities, thus generalizing locomotion in settings where real-world-like hurdles are present.

- Leveraging the independence of our foothold-tracking policy, we show its modularity by pairing it with various high-level planners in simulation. We validate performance on tasks such as goal reaching, cluttered environment navigation, and narrow paths and staircases traversal. In most scenarios, we compare against standard velocity-commanded policy, showing that our approach leads to more efficient and robust solutions.
- We deploy the foothold-tracking policy on the Booster T1 humanoid hardware. By integrating a simple vision-based foothold generator, we show zero-shot sim-to-real transfer, tracking the predefined foothold target successfully. This validates the framework’s potential as a foundational block for complex loco-manipulation systems.

Figure 1 shows the proposed training and deployment pipeline.

II. RELATED WORK

Locomotion Over Sparse Footholds. Locomotion over sparse footholds, where the main complexity arises from the combinatorial nature of foothold selection [6], has been extensively studied as a perceptive locomotion problem using model-based methods [34, 43, 16]. Specifically, the literature has focused on developing hierarchical controllers relying on complex pipelines integrating perception, planning, and

control [10, 15, 25, 26, 9, 28]. While years of research led to high-performing, lightweight, and interpretable approaches [34], these methods often suffer from the common pitfalls of model-based control. Indeed, they are highly sensitive to model discrepancies and violations of modeling assumptions. Such problems are especially exacerbated in real world locomotion tasks due to the difficulty of terrain modeling, potentially resulting in unsafe or unreliable behavior during deployment.

To overcome the limitations of model-based methods, research has shifted toward model-free Reinforcement Learning [RL, 40], which learns control policies directly via interaction without relying on explicit priors. Leveraging domain randomization, these approaches have demonstrated remarkable robustness and generalization, enabling effective deployment on real-world hardware [22, 20, 27, 23, 29, 51, 5, 12, 24, 3]. Typically, they learn velocity-tracking policies [1, 46, 5, 47, 50] paired with high-level vision components [11, 4] whose integration is required in sparse foothold scenarios to identify valid places to step on. A major issue in end-to-end model-free-based methodologies arises from the sparsity of valid footholds which results in sparse reward signals and frequent early episode terminations. To mitigate these limitations, the literature employed curriculum learning [30] and proposed specialized reward structures and architectures, such as a reward account for foot geometry [42] or an attention-based terrain map [11] to achieve robust terrain-aware locomotion.

Another approach to addressing the limitations of both model-based and end-to-end model-free methodologies relies on hybrid architectures that combine the two [48, 8, 45, 31, 17, 14]. The objective is to train agents starting from physics priors [21], aiming to retain the best of both worlds: high interpretability, robustness, and generalization capabilities. Model-based and model-free techniques can be merged in various ways. For

instance, RL has been employed to generate trajectories that are subsequently tracked by model-based controllers [48, 8, 45], or conversely, to train policies capable of tracking model-based planners [17, 14]. While hybrid approaches preserve the benefits of both paradigms, they also inherit their respective pitfalls. Specifically, they rely on models that may fail when discrepancies with reality are significant. Furthermore, they can be computationally demanding and often require substantial implementation effort, particularly when extending established open-source robot locomotion frameworks [e.g., 2].

Learning Foothold-Tracking Controllers. Learning foothold-tracking controllers has gained significant attention in recent years. In most cases, such controllers function as part of two-stage architectures, serving as a bridge between high-level vision pipelines and underlying PD controllers to achieve accurate locomotion on sparse footholds. However, these approaches often rely on unrealistic modeling assumptions, making them unsuitable for real-world deployment. For instance, Peng et al. [32] proposed *DeepLoco*, a two-stage architecture for terrain-aware humanoid locomotion employed solely in simulation. Besides requiring proprioceptive information and a gait phase indicator [37] as input, the tracker additionally requires observing binary ground contact flags which are difficult to estimate accurately in the real world. Furthermore, the foothold goals consist of target positions for the next two steps [49] and a target root heading, all expressed in the root frame, as also done in [39]. This implies that real-world deployment would require precise state estimation/localization. Moreover, during training, foothold targets are extracted from motion capture clips to enforce stylistic mimicry. Despite these limitations, [32] is considered a seminal work, inspiring numerous follow-up studies [41, 17, 11]. Another key study focusing on simulation-based foothold tracking is [44]. The authors demonstrate that employing a carefully designed curriculum and a lightweight foothold sampler facilitates learning a tracker capable of traversing stepping stones and generalizing to continuous terrains. As with [32], the reliance on binary contact states and body-frame targets renders such controllers difficult to deploy in the real world, likely necessitating specific high-level wrappers. Moving toward real-world applicability, Duan et al. [7] extended the work of Xie et al. [44]. During training, they synchronize touchdown events with the gait clock and focus on forward motion on flat terrain. While simple, this approach paves the way for more sophisticated approaches capable of navigating real-world terrains for general-purpose tasks. More recently, Huang et al. [14] proposed an architecture specifically designed for traversing narrow paths in the real world, leveraging a Linear Inverted Pendulum (LIP) model [19]. Then, they integrate a lightweight LiDAR-based perception module to refine the LIP-proposed footholds [38, 18].

While foothold trackers have proven to be valuable for achieving robust and accurate locomotion by enabling full control over foot placement, to the best of our knowledge, they have been employed primarily as subordinate components within specific high-level pipelines. Furthermore, they often lack generality regarding downstream tasks or upstream

gait generators and frequently face challenges in real-world deployment due to the reliance on specific, hard-to-estimate information in their observation space.

III. PROBLEM FORMULATION

In RL, the environment is modeled as a Markov Decision Process [MDP, 35]. An MDP is represented by $\mathcal{M} := (\mathcal{S}, \mathcal{A}, p, r, \rho_0, \gamma, T)$, where $\mathcal{S} \subseteq \mathbb{R}^{d_s}$ and $\mathcal{A} \subseteq \mathbb{R}^{d_a}$ are the measurable state and action spaces; $p : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})^1$ is the transition model, where $p(s'|s, a)$ is the probability density of landing in state $s' \in \mathcal{S}$ by playing action $a \in \mathcal{A}$ in state $s \in \mathcal{S}$; $r : \mathcal{S} \times \mathcal{A} \rightarrow [-R_{\max}, R_{\max}]$ is the reward function, where $r(s, a)$ is the reward the agent gets when playing action a in state s ; $\rho_0 \in \Delta(\mathcal{S})$ is the initial-state distribution; $\gamma \in [0, 1]$ is the discount factor. A trajectory $\tau = (s_{\tau,1}, a_{\tau,1}, \dots, s_{\tau,T}, a_{\tau,T})$ of length $T \in \mathbb{N} \cup \{+\infty\}$ is a sequence of T state-action pairs. In the following, we refer to $\mathcal{T}$ as the set of all trajectories. The discounted return of a trajectory $\tau \in \mathcal{T}$ is $R(\tau) := \sum_{t=1}^T \gamma^{t-1} r(s_{\tau,t}, a_{\tau,t})$.²

Most of the times, especially in robotics, at the interaction instant t the agent is not provided with the full state s_t , but just with a partial observation of it. Specifically, it is provided with $o_t \in \mathcal{O} \subseteq \mathbb{R}^{d_o}$, which can be expressed as $o_t = f(s_t)$, where $f : \mathcal{S} \rightarrow \mathcal{O}$. Furthermore, as commonly done in this setting [42, 14, 17, 11], we frame our problem as a goal-based RL one, meaning that we extend the observation space to incorporate a goal set $\mathcal{G} \subseteq \mathbb{R}^{d_g}$, part of the state space of the MDP. Thus, the goal $g_t \in \mathcal{G}$ influences the reward function r .

An RL agent interacts with the environment according to a parametric stochastic policy $\pi_\zeta : \mathcal{O} \times \mathcal{G} \rightarrow \Delta(\mathcal{A})$, where $\zeta \in \mathcal{Z}$ is the policy parameterization in the parameter space $\mathcal{Z} \subseteq \mathbb{R}^{d_z}$. The policy is employed to sample actions $a_t \sim \pi_\zeta(\cdot | o_t, g_t)$ to be played at every interaction instant t , being conditioned on the observation $o_t \in \mathcal{O}$ and the goal $g_t \in \mathcal{G}$. The performance of a policy π_θ is assessed via the expected return $J : \mathcal{Z} \rightarrow \mathbb{R}$, which is defined as: $J(\zeta) := \mathbb{E}_{\tau \sim p_\zeta} [R(\tau)]$, where $p_\zeta(\tau)$ is the density function of a trajectory τ induced by π_ζ . The goal is to learn the optimal policy π_{ζ^*} , with $\zeta^* \in \arg \max_{\zeta \in \mathcal{Z}} J(\zeta)$.

IV. METHODOLOGY

In this section, we detail the proposed framework. First, we define the foothold command formulation provided as input to the policy and describe the observation space, which is explicitly designed to facilitate real-world deployment. Next, we outline the characteristics of our training pipeline, detailing the employed agent structure and the goal sampler functioning. Finally, we introduce the reward function, focusing on the specific terms for foothold tracking and locomotion style.

A. Goal and Observation Formulation

At each time step t , the policy observes proprioceptive information $o_{\text{proprio}} := (\theta_{\text{base}}, \omega, q, \dot{q})$ (including base orientation θ_{base} , base angular velocity ω , joint positions q and velocities $\dot{q}$) along with a goal command specifying the desired footholds

¹ $\Delta(\mathcal{X})$ is the set of probability distributions over a measurable space $\mathcal{X}$.

²We let $\gamma = 1$ only when $T < +\infty$.

in terms of position and yaw orientation. We formulate the foothold target goal as a vector $\mathbf{g}_t$, defined as:

$$\mathbf{g}_t := ({}^s\mathbf{p}_t; {}^s\psi_t; {}^r\mathbf{p}_t; {}^r\psi_t),$$

where ${}^s\mathbf{p}_t, {}^r\mathbf{p}_t \in \mathbb{R}^3$ denote the position offsets of the next desired left and right footholds, expressed in the *current stance foot frame*. Similarly, ${}^s\psi_t, {}^r\psi_t$ represent the desired orientations (expressed as quaternions) in the *stance foot frame*. As these foothold targets are expressed in the *stance foot frame*, the targets are holding a constant value during each *swing phase*, significantly reducing target variability during movement compared to previous methods [32, 44, 39].

The agent also observes *gait phase information*: $\phi_t := (\cos(2\pi\phi_t); \sin(2\pi\phi_t))$, where $\phi_t \in [0, 1]$. If $\phi_t \in [0, 0.5)$, the left foot acts as the swing foot; otherwise, the right foot swings. As commonly done in this setting [32, 37, 44], we consider the phase variable to evolve like a clock, enforcing a constant locomotion cadence. Specifically, it evolves according to $\phi_{t+1} \leftarrow \phi_t + \delta_\phi \bmod 1$, where δ_ϕ is the *fixed gait offset* determining the rapidity of gait phase switches.

Design Choices. Several aspects of this formulation differ from prior art. First, all targets in $\mathbf{g}_t$ are expressed in the *stance-foot frame*. Consequently, the target remains constant until a phase switch occurs. Notice that the target corresponding to the current stance foot is effectively a zero offset (identity pose) for the whole phase duration. This contrasts with methods using the base frame [32, 44, 14, 39] or the specific foot frame [7]. This choice facilitates real-world deployment by removing the need for precise base state estimation (localization) to update targets in real-time, as the relative command remains constant throughout the swing phase. Second, we *avoid providing binary ground contact flags* [32, 44], which are difficult to estimate reliably on hardware. Finally, by *explicitly commanding 3D positions and yaw orientations*, we ensure *general-purpose navigation capabilities on complex terrain*, surpassing prior works that control only subsets of these dimensions [7, 14]. Indeed, by training agents to track full 3D foothold targets, the humanoid enables omnidirectional locomotion (forward, backward, and lateral) while simultaneously controlling swing foot height and orientation. This comprehensive set of control possibilities allows the agent to execute precise footstep plans and complex walking patterns.

B. Training Architecture and Goal Sampling

For training our policy, we employ asymmetric Proximal Policy Optimization (PPO) [36, 33], a common used model-free RL algorithm in locomotion [3]. We *build upon the open-source library LocoMuJoCo* [2], originally designed for velocity-commanded tasks, adapting its infrastructure to support our foothold-tracking framework.

Agent Structure. As done in asymmetric actor-critic, the actor π_ζ , which is parameterized by a $512 \times 256 \times 128$ network, receives proprioceptive observation as input, whereas the critic network shares the same network structure but takes privileged state as observation. The overall observation of the actor is

$$\mathbf{x}_t := (\mathbf{o}_{\text{proprio},t}; \mathbf{a}_{t-1}; \phi_t; \mathbf{g}_t),$$

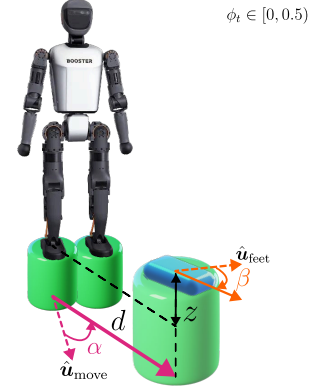


Fig. 2: **Goal Sampling.** Representation of the sampling process during a left-swing phase ($\phi_t < 0.5$). The global vectors $\hat{\mathbf{u}}_{\text{move}}$ and $\hat{\mathbf{u}}_{\text{feet}}$ define the *nominal locomotion heading and foot orientation*, fixed for the entire episode. At each gait switch, the angles α and β are sampled from independent uniform distributions to perturb the nominal directions. The scalars d and z determine the planar distance and vertical offset, respectively, relative to the current stance foot.

where the components are defined in the preceding section. In addition to the actor’s observation, the *critic also observes the linear velocity of the robot base $\mathbf{v}_{\text{base}}$* . Regarding the output action $\mathbf{a}_t \sim \pi_\theta(\cdot | \mathbf{x}_t)$, we consider $\mathbf{a}_t$ to specify the target joint positions for a low-level PD controller, a common hierarchy in the literature [32, 14, 11]. In this work, we target the Booster T1 humanoid, though the formulation is robot-agnostic. Figure 1 illustrates the complete architecture.

Goal Sampling. During training, we employ a Goal Sampler (GS) responsible for generating feasible foothold targets and supports. In the following, we denote quantities in the world frame w unless otherwise specified. We remark that the *GS operates just in simulation, while the agent is then provided with the observation described above*. At the *beginning of each episode* ($t = 1$), the GS defines the global locomotion heading. It samples a desired movement direction $\theta_{\text{move}} \in [-\pi, \pi)$ and a nominal feet orientation $\theta_{\text{feet}} \in [-\pi, \pi)$. These global directions *remain constant throughout the episode to enforce motion consistency*. The gait phase ϕ_1 , defining the start of the motion with either the left or the right foot, is initialized conditioned to θ_{move} or sampled at random whenever possible. *Target sampling is triggered at every gait phase switch* (i.e., whenever the swing and stance roles swap). The GS computes the target position ${}^w\mathbf{p}_{\text{tgt}}$ and yaw orientation ${}^w\psi_{\text{tgt}}$ for the swing foot as follows:

- 1) The GS samples a deviation angle $\alpha \sim \mathcal{U}(\alpha_{\min}, \alpha_{\max})^3$ and a step length $d \sim \mathcal{U}(d_{\min}, d_{\max})$. The planar displacement is computed by perturbing the nominal movement direction θ_{move} :

$$\Delta \mathbf{p} = (d \cos(\theta_{\text{move}} + \alpha); d \sin(\theta_{\text{move}} + \alpha); 0).$$

This ensures that while the robot follows a general direction, it must handle lateral and angular variations.

³ $\mathcal{U}(a, b)$ denotes a uniform distribution continuous in $[a, b]$.

- 2) Similarly, the target foot yaw is generated by perturbing the global feet orientation with an offset $\beta \sim \mathcal{U}(\beta_{\min}, \beta_{\max})$, yielding ${}^w\psi_{\text{tgt}} = \theta_{\text{feet}} + \beta$.
- 3) The vertical component is sampled as an offset $z \sim \mathcal{U}(z_{\min}, z_{\max})$ relative to the current stance foot height. The final world-frame target is:

$${}^w\mathbf{p}_{\text{tgt}} = {}^w\mathbf{p} + \Delta\mathbf{p} + (0; 0; z).$$

Crucially, to ensure physical feasibility in simulation, we implement a *terrain adaptation* mechanism: the environment consists of moving pillars that automatically adjust their height to match the sampled z -coordinate of the target, guaranteeing a valid contact surface.

Finally, the sampled target (${}^w\mathbf{p}_{\text{tgt}}, {}^w\psi_{\text{tgt}}$) is transformed into current stance foot frame and clipped to feasible region to avoid leg crossing (${}^s\mathbf{p}_t, {}^s\psi_t$), where $\square \in \{l, r\}$ depending on the current swing foot index, to build the goal vector $\mathbf{g}_t$ as previously described. We highlight that in $\mathbf{g}_t$ the part related to the stance foot is taken as the offsets in the stance foot frame of the last observed position and orientation of the stance foot itself at the *gait switch instant*. To enable *standing capabilities*, with probability p_{hold} , the GS enters a “hold-still” mode. In this case, the target position is set to maintain a nominal stance width relative to the stance foot, the orientation is aligned with the current stance foot (identity relative quaternion), and the gait phase is frozen providing the agent with $\phi_t = (0; 0)$ for every t in which the GS remains in the still mode. Figure 2 shows the sampling strategy.

Additional Training Details. As previously mentioned, we utilize Asymmetric PPO [36, 33], a standard model-free RL algorithm in robot learning [3]. Specifically, we adopt an asymmetric actor-critic architecture, where the critic has access to privileged simulation states unavailable to the actor. We define an episode as *terminated* (absorbing state) if the robot’s base height falls below a critical threshold. Furthermore, we employ *extensive domain randomization*, including perturbations to link geometries, masses, and contact friction, as well as observation noise and external force disturbances (simulating kicks). Further details are provided in the supplementary material. Finally, we implement a *two-stage training curriculum*. Initially, we train the agent on flat terrain ($z_{\min} = z_{\max} = 0$) to learn a robust planar tracker, then we fine-tune the agent on full 3D terrain.

C. Reward Design

The reward function is mainly composed by tracking reward terms, the responsible ones for making the agent learn foothold-tracking capabilities and penalization terms employed to enforce a natural locomotion style. Penalization terms, which are taken from the velocity-based locomotion reward implemented in *LocoMuJoCo* [2], allow to learn a natural locomotion style, without relying on motion capture-based terms [32]. In this section we focus on tracking terms, while the penalization ones are detailed in appendix.

Foothold-Tracking Terms. The fundamental term of our reward function is represented by the one responsible for tracking the goal $\mathbf{g}_t$. Let (${}^w\bar{\mathbf{p}}_t; {}^w\bar{\psi}_t$), with $\square \in \{l, r\}$, be

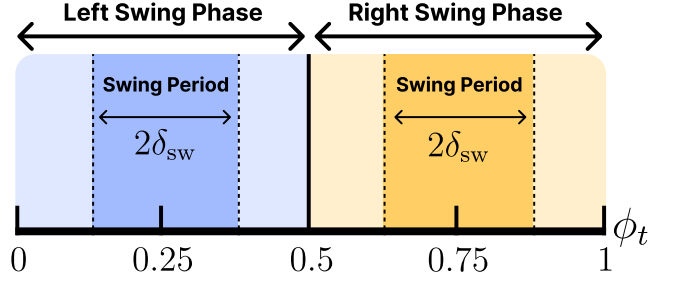


Fig. 3: **Swing-based Reward.** Illustration of the desired swing interval. The reward term r_{feet} incentivizes foot clearance specifically when the phase variable ϕ_t lies within the window $2\delta_{\text{sw}}$. A non-zero reward is accumulated only if the correct foot breaks contact with the ground during this allotted time.

the target foothold positions and orientations to be tracked by both feet. We define the foothold-tracking term as follows:

$$r_{\text{track}}(\mathbf{s}_t, \mathbf{a}_t) = r_{\text{sw}}(\mathbf{s}_t, \mathbf{a}_t) + r_{\text{st}}(\mathbf{s}_t, \mathbf{a}_t),$$

where r_{sw} and r_{st} refer to the swing and stance foot, respectively. Notice that we explicited the fact that this reward term depends on the state $\mathbf{s}_t$ rather than on the observation $\mathbf{o}_t$, since it needs world frame coordinates, always accessible in simulation. Specifically, we define *swing*-related term as:

$$\begin{aligned} r_{\text{sw}}(\mathbf{s}_t, \mathbf{a}_t) = & \omega_1 \exp(-\xi_1 \| {}^w\bar{\mathbf{p}}_{t,xy} - \square\bar{\mathbf{p}}_{t,xy} \|_2^2) \\ & + \omega_2 \exp(-\xi_2 ({}^w\bar{\mathbf{p}}_{t,z} - \square\bar{\mathbf{p}}_{t,z})^2) \\ & + \omega_3 \exp(-\xi_3 ({}^w\bar{\psi}_t - \square\bar{\psi}_t)^2), \end{aligned}$$

where $\square \in \{l, r\}$ depends on the current ϕ_t value, ${}^w\bar{\psi}_t$ and $\square\bar{\psi}_t$ are the yaws (wrapped in $[-\pi, \pi)$) relative to the quaternions ${}^w\bar{\psi}_t$ and $\square\bar{\psi}_t$, and $\bar{\mathbf{p}}_{t,xy}$ and $\bar{\mathbf{p}}_{t,z}$ represent the planar and the height components of vector $\bar{\mathbf{p}}_t$, respectively. The $r_{\text{sw}}(\mathbf{s}_t, \mathbf{a}_t)$ explicitly allows the policy to track the 3D target foothold for the swinging foot. Notice that decoupling planar and height components in $r_{\text{sw}}(\mathbf{s}_t, \mathbf{a}_t)$ allows to define different sharpness coefficients ξ_1 and ξ_2 . In this context, this is needed given the different ranges covered by planar and height components. Finally, the *stance* reward term $r_{\text{st}}(\mathbf{s}_t, \mathbf{a}_t)$ is designed to retain the final value of the swing reward r_{sw} computed at the moment of touchdown. Specifically, for the entire duration of the stance phase, this term equals the accuracy achieved by the foot at the instant it transitioned from swing to stance.

Feet Swing Term. To ensure adherence to the commanded gait schedule, we introduce the term r_{feet} . This reward encourages the feet to lift during the allotted swing phase defined by the variable ϕ_t . Similar swing-phase incentives are standard in humanoid locomotion frameworks [2]. As depicted in Figure 3, we employ a temporal window within each gait phase where foot clearance is mandatory. Formally:

$$\begin{aligned} r_{\text{feet}}(\mathbf{s}_t, \mathbf{a}_t) = & \omega_{\text{feet}} \mathbb{1}\{|\phi_t - 0.25| \leq \delta_{\text{sw}}\} \cdot \mathbb{C}_l \\ & + \omega_{\text{feet}} \mathbb{1}\{|\phi_t - 0.75| \leq \delta_{\text{sw}}\} \cdot \mathbb{C}_r, \end{aligned}$$

where δ_{sw} represents the half-width of the swing window, and $\mathbb{C}_{\square}$ ($\square \in \{l, r\}$) acts as a binary “no-contact” indicator

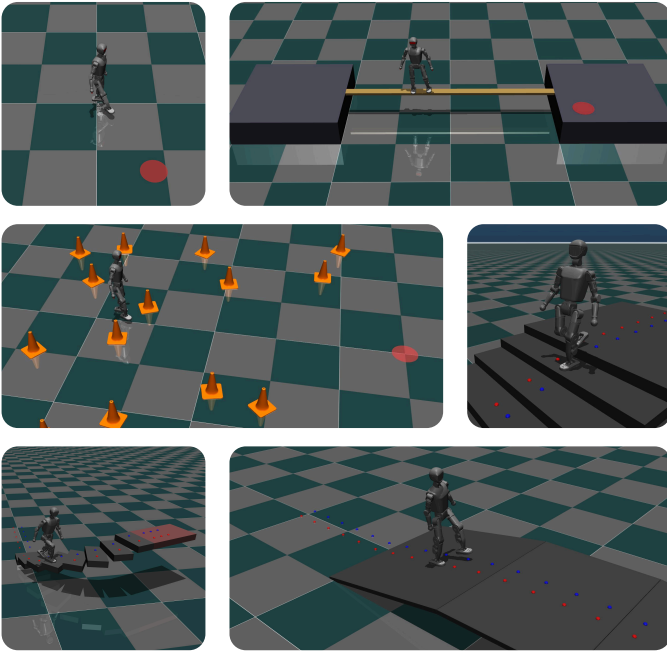


Fig. 4: **Evaluation Tasks in Simulation.** Simulation environments considered in Section V. Evaluations were conducted on: goal reaching, narrow bridge traversal, navigation in cluttered environments, and stairs and ramp climbing.

(returning 1 when the foot is in the air). Also in this case, this term depends on the full state s_t rather than the observation o_t , as it relies on ground truth contact data. Note that, unlike the tracking term, r_{feet} is purely locomotion-related and does not depend on the foothold goal g_t .

Swing Knee Height Term. The last fundamental tracking term encourages the robot to raise the knee of the swing leg, a particularly important skill when tracking the z component of the desired foothold. Specifically, lifting the knee ensures sufficient clearance to bring the swing foot to the target height, avoiding collisions with steps or obstacles. This term is defined exclusively for the swing leg. The reward term is as follows:

$$r_k(s_t, a_t) = \omega_k \exp\left(-\xi_k \max\left(\square^w \bar{p}_{t,z} + \delta_k - \square_{,k}^w p_{t,z}, 0\right)^2\right),$$

where $\square \in \{l, r\}$ depends on the current ϕ_t to select the swing leg, $\square^w \bar{p}_{t,z}$ is the z target for the swing foot in the world frame, δ_k is a slack representing a knee clearance margin, and $\square_{,k}^w p_{t,z}$ is the world-frame z coordinate of the knee joint of the swing leg. This term is one-sided: the reward is maximal whenever the knee exceeds the threshold $\square^w \bar{p}_{t,z} + \delta_k$, and decays otherwise.

V. EXPERIMENTS

We evaluate the effectiveness of the proposed framework in simulation by comparing our Foothold-Tracking (FT) policy (π_{FT}) against a state-of-the-art Velocity-Tracking (VT) baseline (π_{VT}) trained via *LocoMuJoCo* [2]. Due to the discrepancy of the command types, to ensure a fair comparison, both policies are trained within a equivalent command region, where velocity command v_{FT} can be converted by the foot distance v_{VT} and frequency ν as $d_{\text{FT}} v_{\text{VT}} = 2d_{\text{FT}}\nu$. Through

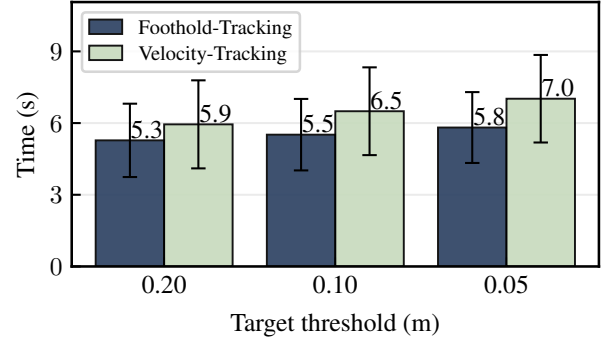


Fig. 5: **Goal-Reaching Experiment.** Time required to reach a target below different thresholds. Mean mean and standard deviation are shown for each setting with 1000 trials.

a comprehensive set of carefully designed experiments, we demonstrate the agility, accuracy, and flexibility of our FT policy, showing that it enables a general-purpose locomotion policy with a wide range of motion capabilities. We use multiple task-specific high-level planners for π_{FT} in each experiment. This highlights the framework’s modularity, demonstrating that the learned policy serves as a general-purpose low-level controller capable of seamless integration with diverse upstream generators. Evaluation environments are illustrated in Figure 4.

A. Goal Reaching Task

We first evaluate the policies on the task of reaching a goal precisely and efficiently, which is an essential requirement for whole-body manipulation tasks. Here, we deploy a variant of π_{FT} trained on flat ground to isolate planar maneuvering performance. Targets are sampled in a $[-3, 3]\text{m}$ region, and task success is defined by multiple distance-based thresholds. Additionally, we compare π_{FT} against $\pi_{\text{FT-R}}$, a baseline identical to our method except that foothold targets are expressed in the *root* frame rather than the *stance foot* frame. This ablation isolates the contribution of the stance-foot frame formulation.

High-Level Planners. For π_{FT} and $\pi_{\text{FT-R}}$, we implement a simple planner that generates footholds along the vector connecting the robot’s base position to the goal. The step length is clamped to a maximum of 0.5 m, and footholds are constrained to feasible regions to prevent leg crossing. In the meantime, the foot orientation is steered towards the target direction to encourage forward walking. Footholds are generated respecting the gait frequency the policy saw during training. For π_{VT} , we employ a standard proportional controller: the target velocity is computed as $v_{\text{cmd}} = K(p_{\text{goal}} - p_{\text{base}})$, saturated to the robot’s maximum velocity limits. The angular velocity is controlled by a separate P-controller to maintain the heading towards the target. In both methods, we assume the base positions are directly observable. Although unrealistic for real-world deployments, it allows us to decouple the effect from the localization errors encountered by both methods.

FT vs. VT. We quantify performance by measuring the time-to-goal under three distinct acceptance radii, specifically, 0.2m, 0.1m, and 0.05m. For each threshold, we evaluate 1,000

Policy (σ_{xyz} (m), σ_{yaw} (deg))	FT (Ours) —	FT-R (0, 0)	FT-R ($3e-2$, 2.0)	FT-R ($6e-2$, 3.0)
Accuracy (cm) (mean $\pm$ std)	2.15 ± 0.164	1.66 ± 0.170	2.07 ± 0.237	2.86 ± 0.264

TABLE I: **Frame Ablation.** Foothold-tracking accuracy across 1000 targets on flat terrain, comparing π_{FT} and π_{FT-R} . σ_{xyz} (m) and σ_{yaw} (deg) are the position and yaw noise scales for estimating base estimation errors for π_{FT-R} .

trials with randomly sampled target positions. As shown in Figure 5, π_{FT} consistently outperforms π_{VT} , reaching the target significantly faster across all acceptance radii. This performance gain exhibit higher agility of the foot-tracking policy due to explicit control of the feet. Notably, as the success threshold decreases, the time-to-goal increases for π_{FT} by only 5.3%, whereas π_{VT} exhibits a substantially larger increase of 18.9%. This degradation for π_{FT} majorly stems from the inherit mismatch between the continuous velocity commands and discretized footstep execution, which necessitates multiple correcting steps near the target. On the contrary, foothold-tracking directly command next foothold based on its current state and is therefore limited mainly by tracking accuracy. The results validate that explicit foothold control enables more aggressive and precise maneuvering compared to velocity-based commands, further narrowing the gap between high-level planning and low-level execution.

Frame Ablation. Table I reports foothold-tracking accuracy across 1000 targets on flat terrain. To simulate real-world deployment conditions, we inject uniform noise into the base state estimate of π_{FT-R} , considering two noise scales to cover a range from mild to realistic localization error. Without noise, π_{FT-R} improves over π_{FT} in tracking accuracy, as the root frame provides a precise target signal under ideal conditions. However, performance degrades sharply even under small noise magnitudes, demonstrating that root-frame policies are sensitive to localization errors. By contrast, π_{FT} is unaffected by state estimation noise by design, validating the stance-foot frame as the more robust formulation for real-world deployment.

B. Narrow Bridge Traversal

In this experiment, we challenge motion accuracy between two tracking policies (π_{FT} and π_{VT}) in a narrow bridge traversal task as shown in Fig 4-UpperRight. This scenario demands high-precision foot placement, as the tolerance for error is extremely low. The robot is initialized at a different location, aiming to pass a bridge with different widths and lengths.

High-Level Planners. Both agents employ similar planners introduced in the goal-reaching experiment, with additional task-specific modifications. In particular, auxiliary waypoints are placed at the entrance and exit of the bridge to enforce the robot moving parallel to the bridge’s longitudinal direction. Due to the bridge’s restricted width, successful traversal requires precise lateral control. For π_{FT} , the planner generates foothold targets that explicitly maintain the robot’s footsteps near the centerline of the bridge. For π_{VT} , we increase the control gain on the robot’s heading direction to correct deviations that could otherwise result in a fall.

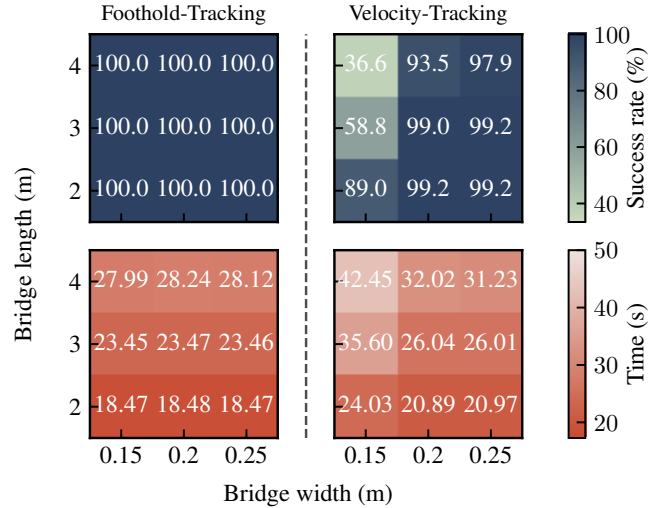


Fig. 6: **Narrow Bridge Experiment.** The success rate(↑) and traversing time(↓) to cross a narrow bridge of different widths and lengths. Bridge width varies from 0.15 to 0.25m and length varies from 2 to 4m. Each configuration is conducted with 1000 trials with randomly sampled initial states and targets.

# Cones	Success (%)	Fall (%)	Plan fail (%)
10	99	1	0
15	98	0	2
20	90	0	10

TABLE II: **Cluttered Environment Experiment.** 100 episodes in a 6 m $\times$ 6 m workspace with randomly placed cones.

Results. We evaluate performance by measuring both success rate and traversal time across a range of bridge widths (0.15–0.25 m) and lengths (2–4 m). While both methods achieve high success rates, for less challenging configurations with a wider and shorter bridge, our π_{FT} controller consistently completes the traversal in less time, indicating more efficient motion execution. As the task difficulty increases, in the critical scenarios with a width of 0.15m, narrower than the robot’s foot length, the success rates for the VT controller drops drastically, whereas the FT-controller maintains almost the same performance. This is because the FT-tracking explicitly sets the desired target to be at the center of the bridge, eliminating the errors derived from proportional controller. Our foothold tracking controller provides a transparent and interpretable control interface enabling precise step-level reasoning and correction. This capability is crucial for safety-critical locomotion tasks, where accurate contact placement is required to prevent catastrophic failures, demonstrating the suitability of foothold-based control for real-world, constrained navigation scenarios.

C. Traversing Complex Environments

Finally, we demonstrate the integration of the trained π_{FT} in four additional tasks: cluttered environment traversal, straight and spiral staircase climbing, and ramp traversal.

Traversing Cluttered Environments. Here the humanoid

Step Length (m)	Step Height (m)						
	0.08	0.10	0.11	0.115	0.12	0.125	0.13
0.300	95% R:3 B:2 0.059±0.089	76% R:19 B:5 0.071±0.049	60% R:40 B:0 0.081±0.048	52% R:34 B:14 0.090±0.048	56% R:39 B:5 0.083±0.041	38% R:47 B:15 0.095±0.038	12% R:70 B:18 0.108±0.039
	100% R:0 B:0 0.038±0.005	100% R:0 B:0 0.041±0.006	92% R:5 B:3 0.050±0.029	89% R:11 B:0 0.053±0.027	74% R:22 B:4 0.068±0.042	30% R:54 B:16 0.104±0.045	6% R:80 B:14 0.115±0.038
0.325	100% R:0 B:0 0.040±0.005	99% R:1 B:0 0.045±0.022	100% R:0 B:0 0.045±0.016	96% R:2 B:2 0.050±0.022	89% R:7 B:4 0.066±0.056	47% R:34 B:19 0.106±0.060	19% R:58 B:23 0.136±0.057
	100% R:0 B:0 0.048±0.011	100% R:0 B:0 0.052±0.010	99% R:0 B:1 0.054±0.017	86% R:4 B:10 0.058±0.019	88% R:8 B:4 0.079±0.100	87% R:6 B:7 0.087±0.112	46% R:23 B:31 0.113±0.086

(a) Straight Stairs.

Step Length (m)	Step Height (m)						
	0.08	0.10	0.11	0.115	0.12	0.125	0.13
0.300	97% R:2 B:1 0.045±0.017	82% R:17 B:1 0.061±0.029	40% R:40 B:20 0.082±0.028	21% R:61 B:18 0.104±0.031	14% R:71 B:15 0.114±0.034	19% R:73 B:8 0.112±0.030	14% R:72 B:14 0.132±0.053
	97% R:2 B:1 0.049±0.044	91% R:3 B:6 0.060±0.025	77% R:15 B:8 0.066±0.035	58% R:20 B:22 0.085±0.040	66% R:31 B:3 0.082±0.050	29% R:59 B:12 0.097±0.038	17% R:62 B:21 0.104±0.031
0.350	96% R:0 B:4 0.061±0.068	94% R:4 B:2 0.065±0.088	91% R:3 B:6 0.074±0.098	84% R:8 B:8 0.084±0.095	79% R:13 B:8 0.093±0.107	57% R:25 B:18 0.097±0.059	28% R:55 B:17 0.132±0.061
	91% R:4 B:5 0.081±0.038	90% R:4 B:6 0.090±0.137	96% R:1 B:3 0.069±0.048	93% R:1 B:6 0.089±0.126	74% R:7 B:19 0.097±0.067	54% R:16 B:30 0.125±0.107	17% R:45 B:37 0.155±0.097

(b) Spiral Stairs.

TABLE III: **Traversing Straight and Spiral Stairs.** Each cell reports success rate, riser-contact falls (R) and balance falls (B), and mean $\pm$ std 3D foot placement error (m). 100 evaluations with initial robot yaw sampled from $\mathcal{U}(-\pi/6, \pi/6)$.

is tasked with navigating a 6×6 m region with up to 20 traffic cones, as shown in Figure 4. We employ Anytime Repairing A* (ARA*) planner [13] to generate a sequence of foothold targets. The target candidates are selected from a set defined within the policy’s training range. We evaluate scenarios with 10, 15, and 20 traffic cones, conducting 100 trials for each configuration. Table II shows that our π_{FT} integrates seamlessly with the planner across all tested cases, with only one falling case observed. Failures are primarily due to the planning algorithms not finding a feasible path within the time budget.

Traversing Straight Stairs. We evaluate the 3D-trained π_{FT} on a structured obstacle comprising an ascending staircase (5 steps), a landing platform, and a descending staircase (5 steps). π_{FT} was trained with elevation targets sampled in $[-0.15, 0.15]$ m. We analyze performance over 100 episodes, randomizing the initial yaw orientation in $\mathcal{U}(-\pi/6, \pi/6)$ and varying stair geometry (step length and height). A deterministic planner feeds π_{FT} with 3D footholds matching the terrain profile. Table IIIa shows that as the step height increases, the step length becomes a decisive factor: shorter steps leave little margin for tracking errors, as the swing foot must lift nearly vertically to clear the riser, making even minor inaccuracies likely to cause collisions. Longer steps provide a greater safety margin, allowing the policy to tolerate small placement imperfections without contacting the subsequent riser. This is further confirmed by the failure mode breakdown. At shorter step lengths, failures are dominated by riser contacts, reflecting the tight clearance constraints. As step length enlarges, riser-contact failures decrease and balance-related falls become the primary failure mode, a consequence of the wider stance required to sustain a longer gait cycle.

Traversing Spiral Stairs. We further evaluate the 3D-trained π_{FT} on a more challenging terrain consisting of an ascending staircase with rotating steps (-10° yaw offset per step) terminating on a flat platform, designed to emulate a spiral staircase and to validate the omnidirectional capabilities of policies trained within the proposed framework. The policy is identical to that used in the straight staircase experiment. Table IIIb shows that the results mirror those of the straight staircase: longer steps consistently reduce riser-contact failures. However, due to the step rotation, targets are no longer placed at uniform lateral and vertical distances, meaning that further

Step Length (m)	Final Rise (m)						
	0.05	0.10	0.15	0.20	0.25	0.30	0.35
0.300	100% B:0 0.039±0.009	100% B:0 0.046±0.007	99% B:1 0.058±0.015	76% B:24 0.133±0.093	47% B:53 0.155±0.085	7% B:93 0.319±0.172	0% B:100 0.350±0.154
	100% B:0 0.037±0.004	100% B:0 0.043±0.009	99% B:1 0.057±0.015	96% B:4 0.092±0.071	87% B:13 0.120±0.060	35% B:65 0.230±0.140	5% B:95 0.338±0.151
0.350	100% B:0 0.040±0.011	100% B:0 0.045±0.010	97% B:3 0.064±0.049	96% B:4 0.090±0.051	89% B:11 0.110±0.057	68% B:32 0.199±0.113	6% B:94 0.380±0.170
	100% B:0 0.054±0.019	100% B:0 0.065±0.016	95% B:5 0.104±0.060	85% B:15 0.115±0.064	47% B:53 0.173±0.065	46% B:54 0.207±0.101	57% B:43 0.201±0.103

TABLE IV: **Traversing Ramps.** Each cell reports success rate, balance falls (B), and mean $\pm$ std 3D foot placement error (m). 100 evaluations with initial robot yaw $\sim \mathcal{U}(-\pi/6, \pi/6)$.

increasing the step length does not monotonically reduce failures. As evidenced by the results, the optimal step length is 0.35 m, beyond which balance-related failures begin to increase. Together with the straight staircase experiment, these results confirm that π_{FT} is capable of complex omnidirectional locomotion while operating without terrain perception, relying solely on proprioception.

Traversing Ramps. Finally, we evaluate π_{FT} on an additional obstacle, structurally analogous to the straight staircase scenario, but replacing discrete steps with continuous ascending and descending ramps of varying inclination. This experiment is critical for assessing the generalization capabilities of a policy trained within the proposed framework. Indeed, as described in Section IV, the 3D fine-tuning stage employs moving pillars as contact surfaces, meaning the policy never encounters sloped foot placements during training. Successfully traversing ramps therefore requires the policy to generalize its z -tracking capabilities to unseen foot pitch configurations. Table IV shows that π_{FT} handles this setting effectively. A trend consistent with the staircase experiments emerges: longer steps yield higher success rates as task difficulty increases. However, this phenomenon has a different explanation here. At high inclinations with short steps, the robot loses balance due to the steep vertical rise demanded per foothold. Conversely, excessively long steps also increase balance failures, as the robot must sustain a longer gait cycle. The optimal step length sits around 0.35 m across most inclinations. These results confirm that the proposed framework produces policies with strong generalization capabilities.

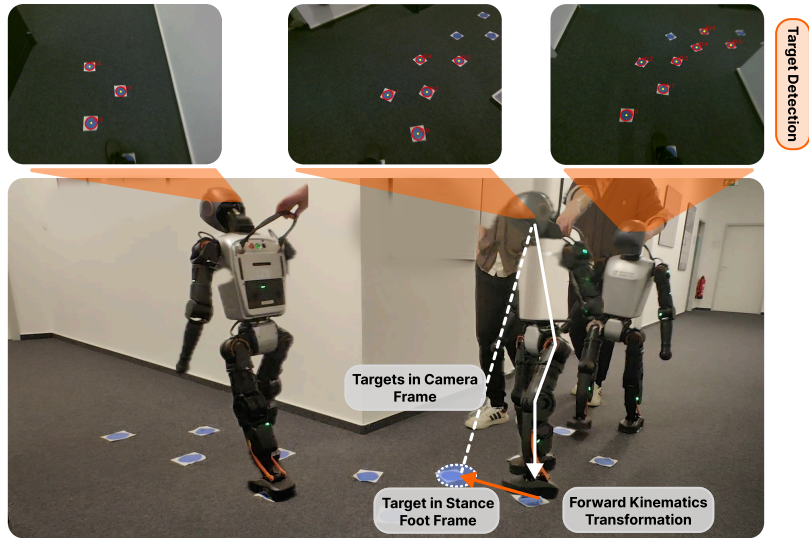


Fig. 7: **Real-World Deployment.** Sequence of the robot tasked with traversing a corridor while stepping on predefined targets. Markers are detected via the onboard camera and then mapped to valid footholds targets in the stance foot frame via a transformation by forward kinematics. For each pose snapshot, we display the corresponding camera frame with highlighted targets. The second pose from the right details the target generation logic during a gait phase switch. Here, the right foot becomes the new stance foot.

VI. REAL-WORLD RESULTS

In the real-world experiment, to avoid additional complexity introduced by the localization problem, we employ a vision-based foothold tracking task to verify our FT policy. As illustrated in Figure 7, a sequence of blue circular markers is manually placed on the ground in a corner of a corridor to serve as visual foothold targets. We use the onboard RealSense D455 camera of the Booster T1 robot to extract the targets. By fusing RGB-based target detection, the depth measurements, and the robot’s kinematic model, we compute the relative transformations between the targets and the robot’s current stance foot. The foothold orientations are heuristically determined based on the upcoming swing targets to ensure the feasibility of the future targets. A snapshot of the target detection results is illustrated in Figure 7. Due to delays introduced by camera frame rate limitations, communication latency, and policy inference time, the swing foot may not reach the commanded foothold exactly at the nominal swing–stance transition defined by the gait phase. To address this issue, we implement a simple delay compensation mechanism that repeats the same gait phase ϕ_t and foothold target g_t until the relative height between the swing and the stance foot falls below a threshold, ensuring the relative targets are computed with the next stance on the ground.

Results. We evaluate the system over 20 real-world trials, including both forward and backward traversals in the corridor with manual resets. An uncut video of the experiments is provided as supplementary. Despite sensing noise and system delays, the robot achieves 93.08% success of stepping on designated foothold targets while maintaining stable locomotion. These results demonstrate the practical viability of the proposed FT controller in real-world settings, even when driven by

imperfect visual perception. The primary sources of foot-placement error arise from motion-induced image blur, which degrades target detection accuracy, and from depth sensing artifacts, as the depth camera is not optimized for highly dynamic motion. Furthermore, although the sim-to-real gap is significantly mitigated by the large-scale domain randomization, it also poses costs in losing tracking accuracy. Despite the presence of tracking error, this experiment successfully verifies that π_{FT} can be seamlessly integrated into sophisticated high-level planners to support various downstream tasks.

VII. CONCLUSION

In this work, we present a general foothold-conditioned training framework that integrates seamlessly with a wide range of high-level planning algorithms. By introducing a general-purpose 3D goal sampler, our approach enables adaptive terrain generation and supports stable foot placement. To address the limitations of prior methods that rely on accurate pose estimation or explicit foot–contact estimation, we represent foothold targets in the stance-foot frame and hold them constant throughout each swing phase. This design ensures consistent target tracking and robust execution. Through extensive comparisons with a standard velocity-tracking policy, we show that our foothold tracking controller achieves more agile and efficient goal reaching, delivers consistent performance in precision-critical tasks, and robustly traverses challenging cluttered environments and staircases. Finally, we validate the real-world deployability of our approach through a fully vision-based foothold tracking experiment, demonstrating reliable sim-to-real transfer without reliance on external localization. Together, these results highlight the practicality, versatility, and robustness of the proposed framework for humanoid locomotion in complex safety-critical scenarios.

ACKNOWLEDGMENTS

This publication was funded with the contribution of Ministero dell’Università e della ricerca pursuant to D.D. n. 7206 of 17 April 2025 – BANDO FIS 2. Project FIS-2023-02598 (Starting Grant), title: “Unified Learning from Diverse Human Feedback” (HUmLrn). CUP: D53C25000710001. The project is supported by the Fundamental Research Funds for the Central Universities.

REFERENCES

- [1] Ananye Agarwal, Ashish Kumar, Jitendra Malik, and Deepak Pathak. Legged locomotion in challenging terrains using egocentric vision. In *Conference on robot learning*, pages 403–415. PMLR, 2023.
- [2] Firas Al-Hafez, Guoping Zhao, Jan Peters, and Davide Tateo. Locomujoco: A comprehensive imitation learning benchmark for locomotion. In *6th Robot Learning Workshop, NeurIPS*, 2023.
- [3] Lingfan Bao, Joseph Humphreys, Tianhu Peng, and Chengxu Zhou. Deep reinforcement learning for bipedal locomotion: A brief survey. *arXiv preprint arXiv:2404.17070*, 2024.
- [4] Qingwei Ben, Botian Xu, Kailin Li, Feiyu Jia, Wentao Zhang, Jingping Wang, Jingbo Wang, Dahua Lin, and Jiangmiao Pang. Gallant: Voxel grid-based humanoid locomotion and local-navigation across 3d constrained terrains. *arXiv preprint arXiv:2511.14625*, 2025.
- [5] Xuxin Cheng, Kexin Shi, Ananye Agarwal, and Deepak Pathak. Extreme parkour with legged robots. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 11443–11450. IEEE, 2024.
- [6] Thomas Corbères, Carlos Mastalli, Wolfgang Merkt, Jaehyun Shim, Ioannis Havoutis, Maurice Fallon, Nicolas Mansard, Thomas Flayols, Sethu Vijayakumar, and Steve Tonneau. Perceptive locomotion through whole-body mpc and optimal region selection. *IEEE Access*, 2025.
- [7] Helei Duan, Ashish Malik, Jeremy Dao, Aseem Saxena, Kevin Green, Jonah Siekmann, Alan Fern, and Jonathan Hurst. Sim-to-real learning of footstep-constrained bipedal dynamic walking. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 10428–10434. IEEE, 2022.
- [8] Siddhant Gangapurwala, Mathieu Geisert, Romeo Orsolino, Maurice Fallon, and Ioannis Havoutis. Rloc: Terrain-aware legged locomotion using reinforcement learning and optimal control. *IEEE Transactions on Robotics*, 38(5):2908–2927, 2022.
- [9] Ruben Grandia, Fabian Jenelten, Shaohui Yang, Farbod Farshidian, and Marco Hutter. Perceptive locomotion through nonlinear model-predictive control. *IEEE Trans. Robotics*, 39(5):3402–3421, 2023.
- [10] Robert J. Griffin, Georg Wiedebach, Stephen McCrory, Sylvain Bertrand, Inho Lee, and Jerry E. Pratt. Footstep planning for autonomous walking over rough terrain. In *19th IEEE-RAS International Conference on Humanoid Robots, Humanoids 2019, Toronto, ON, Canada, October 15-17, 2019*, pages 9–16. IEEE, 2019.
- [11] Junzhe He, Chong Zhang, Fabian Jenelten, Ruben Grandia, Moritz Bächer, and Marco Hutter. Attention-based map encoding for learning generalized legged locomotion. *Science Robotics*, 10(105):eadv3604, 2025.
- [12] Tairan He, Chong Zhang, Wenli Xiao, Guanqi He, Changliu Liu, and Guanya Shi. Agile but safe: Learning collision-free high-speed legged locomotion. *arXiv preprint arXiv:2401.17583*, 2024.
- [13] Armin Hornung, Andrew Dornbush, Maxim Likhachev, and Maren Bennewitz. Anytime search-based footstep planning with suboptimality bounds. In *2012 12th IEEE-RAS International Conference on Humanoid Robots (Humanoids 2012)*, pages 674–679. IEEE, 2012.
- [14] TianChen Huang, Runchen Xu, Yu Wang, Wei Gao, and Shiwu Zhang. Traversing narrow paths: A two-stage reinforcement learning framework for robust and safe humanoid walking. *arXiv preprint arXiv:2508.20661*, 2025.
- [15] Fabian Jenelten, Takahiro Miki, Aravind E Vijayan, Marko Bjelonic, and Marco Hutter. Perceptive locomotion in rough terrain—online foothold optimization. *IEEE Robotics and Automation Letters*, 5(4):5370–5376, 2020.
- [16] Fabian Jenelten, Ruben Grandia, Farbod Farshidian, and Marco Hutter. Tamols: Terrain-aware motion optimization for legged systems. *IEEE Transactions on Robotics*, 38(6):3395–3413, 2022.
- [17] Fabian Jenelten, Junzhe He, Farbod Farshidian, and Marco Hutter. Dtc: Deep tracking control. *Science Robotics*, 9(86):eadh5401, 2024.
- [18] Tobias Johannink, Shikhar Bahl, Ashvin Nair, Jianlan Luo, Avinash Kumar, Matthias Loskyll, Juan Aparicio Ojea, Eugen Solowjow, and Sergey Levine. Residual reinforcement learning for robot control. In *2019 international conference on robotics and automation (ICRA)*, pages 6023–6029. IEEE, 2019.
- [19] Shuji Kajita, Fumio Kanehiro, Kenji Kaneko, Kiyoshi Fujiwara, Kensuke Harada, Kazuhito Yokoi, and Hirohisa Hirukawa. Biped walking pattern generation by using preview control of zero-moment point. In *2003 IEEE international conference on robotics and automation (Cat. No. 03CH37422)*, volume 2, pages 1620–1626. IEEE, 2003.
- [20] Ashish Kumar, Zipeng Fu, Deepak Pathak, and Jitendra Malik. Rma: Rapid motor adaptation for legged robots. *arXiv preprint arXiv:2107.04034*, 2021.
- [21] Ho Jae Lee, Seungwoo Hong, and Sangbae Kim. Integrating model-based footstep planning with model-free reinforcement learning for dynamic legged locomotion. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 11248–11255. IEEE, 2024.
- [22] Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. Learning quadrupedal locomotion over challenging terrain. *Science robotics*, 5(47):

eabc5986, 2020.

- [23] Junfeng Long, Zirui Wang, Quanyi Li, Jiawei Gao, Liu Cao, and Jiangmiao Pang. Hybrid internal model: Learning agile legged locomotion with simulated robot response. *arXiv preprint arXiv:2312.11460*, 2023.
- [24] Gabriel B Margolis, Ge Yang, Kartik Paigwar, Tao Chen, and Pulkit Agrawal. Rapid locomotion via reinforcement learning. *The International Journal of Robotics Research*, 43(4):572–587, 2024.
- [25] Carlos Mastalli, Ioannis Havoutis, Michele Focchi, Darwin G Caldwell, and Claudio Semini. Motion planning for quadrupedal locomotion: Coupled planning, terrain mapping, and whole-body control. *IEEE Transactions on Robotics*, 36(6):1635–1648, 2020.
- [26] Oliwier Melon, Romeo Orsolino, David Surovik, Mathieu Geisert, Ioannis Havoutis, and Maurice Fallon. Receding-horizon perceptive trajectory optimization for dynamic legged locomotion with learned initialization. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9805–9811. IEEE, 2021.
- [27] Takahiro Miki, Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. Learning robust perceptive locomotion for quadrupedal robots in the wild. *Science robotics*, 7(62):eabk2822, 2022.
- [28] Bhavyansh Mishra, Duncan Calvert, Sylvain Bertrand, Jerry Pratt, Hakki Erhan Sevil, and Robert Griffin. Efficient terrain map using planar regions for footstep planning on humanoid robots. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 8044–8050. IEEE, 2024.
- [29] I Nahrendra, Byeongho Yu, and Hyun Myung. Dreamwaq: Learning robust quadrupedal locomotion with implicit terrain imagination via deep reinforcement learning. *arXiv preprint arXiv:2301.10602*, 2023.
- [30] Sanmit Narvekar, Bei Peng, Matteo Leonetti, Jivko Sinapov, Matthew E Taylor, and Peter Stone. Curriculum learning for reinforcement learning domains: A framework and survey. *Journal of Machine Learning Research*, 21(181):1–50, 2020.
- [31] Shafeef Omar, Lorenzo Amatiucci, Victor Barasuol, Giulio Turrisi, and Claudio Semini. Safesteps: Learning safer footstep planning policies for legged robots via model-based priors. In *2023 IEEE-RAS 22nd International Conference on Humanoid Robots (Humanoids)*, pages 1–8. IEEE, 2023.
- [32] Xue Bin Peng, Glen Berseth, KangKang Yin, and Michiel Van De Panne. Deeploco: Dynamic locomotion skills using hierarchical deep reinforcement learning. *Acm transactions on graphics (tog)*, 36(4):1–13, 2017.
- [33] Lerrel Pinto, Marcin Andrychowicz, Peter Welinder, Wojciech Zaremba, and Pieter Abbeel. Asymmetric actor critic for image-based robot learning. In *Proceedings of Robotics: Science and Systems*, Pittsburgh, Pennsylvania, June 2018. doi: 10.15607/RSS.2018.XIV.008.
- [34] Jerry Pratt, John Carff, Sergey Drakunov, and Ambarish Goswami. Capture point: A step toward humanoid push recovery. In *2006 6th IEEE-RAS international conference on humanoid robots*, pages 200–207. Ieee, 2006.
- [35] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley Series in Probability and Statistics. John Wiley & Sons, 2014.
- [36] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [37] Jonah Siekmann, Yesh Godse, Alan Fern, and Jonathan Hurst. Sim-to-real learning of all common bipedal gaits via periodic reward composition. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 7309–7315. IEEE, 2021.
- [38] Tom Silver, Kelsey Allen, Josh Tenenbaum, and Leslie Kaelbling. Residual policy learning. *arXiv preprint arXiv:1812.06298*, 2018.
- [39] Rohan P Singh, Mehdi Benallegue, Mitsuharu Morisawa, Rafael Cisneros, and Fumio Kanehiro. Learning bipedal walking on planned footsteps for humanoid robots. In *2022 IEEE-RAS 21st International Conference on Humanoid Robots (Humanoids)*, pages 686–693. IEEE, 2022.
- [40] Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.
- [41] Vassilios Tsounis, Mitja Alge, Joonho Lee, Farbod Farshidian, and Marco Hutter. Deepgait: Planning and control of quadrupedal gaits using deep reinforcement learning. *IEEE Robotics and Automation Letters*, 5(2): 3699–3706, 2020.
- [42] Huayi Wang, Zirui Wang, Junli Ren, Qingwei Ben, Tao Huang, Weinan Zhang, and Jiangmiao Pang. Beamdojo: Learning agile humanoid locomotion on sparse footholds. *Robotics: Science and Systems (RSS)*, 2025.
- [43] Alexander W Winkler, C Dario Bellicoso, Marco Hutter, and Jonas Buchli. Gait and trajectory optimization for legged systems through phase-based end-effector parameterization. *IEEE Robotics and Automation Letters*, 3(3):1560–1567, 2018.
- [44] Zhaoming Xie, Hung Yu Ling, Nam Hee Kim, and Michiel Van De Panne. Allsteps: curriculum-driven learning of stepping stone skills. In *Computer Graphics Forum*, volume 39, pages 213–224. Wiley Online Library, 2020.
- [45] Zhaoming Xie, Xingye Da, Buck Babich, Animesh Garg, and Michiel van de Panne. Glide: Generalizable quadrupedal locomotion in diverse environments with a centroidal model. In *International workshop on the algorithmic foundations of robotics*, pages 523–539. Springer, 2022.
- [46] Ruihan Yang, Ge Yang, and Xiaolong Wang. Neural volumetric memory for visual locomotion control. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 1430–1440, 2023.
- [47] Ruiqi Yu, Qianshi Wang, Yizhen Wang, Zhicheng Wang, Jun Wu, and Qiuguo Zhu. Walking with terrain reconstruction: Learning to traverse risky sparse footholds. *arXiv*

preprint arXiv:2409.15692, 2024.

- [48] Wenhao Yu, Deepali Jain, Alejandro Escontrela, Atil Iscen, Peng Xu, Erwin Coumans, Sehoon Ha, Jie Tan, and Tingnan Zhang. Visual-locomotion: Learning to walk on complex terrains with vision. In *5th Annual Conference on Robot Learning*, 2021.
- [49] Petr Zaytsev, S Javad Hasaneini, and Andy Ruina. Two steps is enough: No need to plan far ahead for walking balance. In *2015 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6295–6300. IEEE, 2015.
- [50] Chong Zhang, Nikita Rudin, David Hoeller, and Marco Hutter. Learning agile locomotion on risky terrains. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 11864–11871. IEEE, 2024.
- [51] Ziwen Zhuang, Zipeng Fu, Jianren Wang, Christopher Atkeson, Soeren Schwertfeger, Chelsea Finn, and Hang Zhao. Robot parkour learning. *arXiv preprint arXiv:2309.05665*, 2023.

APPENDIX

Here, we present additional details for the experiments presented in the main paper. All the media reporting videos for the real-world experiment and the simulated ones can be found in the attached zip and in the project website (<https://montenegroalessandro.github.io/mind-your-steps/>).

A. Observations for Actor and Critic Networks

The table below details the observations received by the actor and critic networks, specifying whether the components are subject to noise.

Network	Observation	Noise
Actor	Base Angular Velocity	✓
	Projected Gravity	✓
	Joint Positions	✓
	Joint Velocities	✓
	Last Action	✗
	Goal	✗
Critic	Base Linear Velocity	✗
	Base Angular Velocity	✗
	Projected Gravity	✗
	Joint Positions	✗
	Joint Velocities	✗
	Last Action	✗
	Goal	✗

B. Domain Randomization

We detail the domain randomization parameters below. Randomization is applied at the beginning of each episode, with the exception of external disturbances, which occur at each timestep with a certain probability. For each parameter, we define the randomization method as follows:

- **Direct:** The value is uniformly sampled from the specified interval and directly employed.
- **Multiplier:** A scaling factor is uniformly sampled from the reported range and multiplied by the nominal value.
- **Additive $\mathcal{U}$:** An offset is sampled uniformly from the range and added to the nominal value.
- **Additive $\mathcal{N}$:** Noise is sampled from a Normal distribution with the specified scale and added to the nominal value.

Parameter	Type	Value / Range
<i>Global Dynamics & Inertial Properties</i>		
Gravity (z -axis)	Direct	$[9.51, 10.11] \text{ m/s}^2$
Root Body Mass	Multiplier	$[0.8, 1.2]$
Link Mass	Multiplier	$[0.9, 1.1]$
CoM Displacement (x, y, z)	Additive $\mathcal{U}$	$[-0.05, 0.05] \text{ m}$
<i>Terrain Contact Properties (Floor & Pillars)</i>		
Tangential Friction	Direct	$[0.5, 1.5]$
Torsional Friction	Direct	$[0.1, 0.3]$
Rolling Friction	Direct	$[8 \times 10^{-5}, 1.2 \times 10^{-4}]$
<i>Actuation & Joint Properties</i>		
Joint Damping	Direct	$[0.005, 0.03] \text{ Nms/rad}$
Joint Friction Loss	Direct	$[0.0, 1.0] \text{ Nm}$
Joint Armature	Direct	$[0.007, 0.03] \text{ kg m}^2$
KP Gain Noise	Additive $\mathcal{U}$	$[-0.15 \cdot K_P, 0.15 \cdot K_P]$
KD Gain Noise	Additive $\mathcal{U}$	$[-0.5 \cdot K_D, 0.5 \cdot K_D]$
<i>Observation Noise</i>		
Joint Position	Additive $\mathcal{N}$	$\pm 0.03 \text{ rad}$
Joint Velocity	Additive $\mathcal{N}$	$\pm 0.3 \text{ rad/s}$
Angular Velocity	Additive $\mathcal{N}$	$\pm 0.2 \text{ rad/s}$
Projected Gravity Vector	Additive $\mathcal{N}$	± 0.015
<i>External Disturbances</i>		
Random Kicks (Velocity)	Additive $\mathcal{U}$ (Per-step w.p. 0.004)	$[0.1, 0.4] \text{ m/s}$

C. PPO Common Training Hyperparameters

The table below details the PPO hyperparameters common to both the flat-terrain and z -tracking training configurations.

Parameter	Value
<i>PPO Algorithm</i>	
Discount Factor (γ)	0.995
GAE Parameter (λ)	0.95
Clipping Range (ϵ)	0.2
Entropy Coefficient	0.01
Value Function Coefficient	0.5
Max Gradient Norm	1.0
<i>Adaptive Learning Rate</i>	
Initial Learning Rate	1×10^{-5}
Min Learning Rate	1×10^{-6}
Max Learning Rate	0.01
KL Target	0.02
KL Margin	1.5
KL Scale Factor	1.5
<i>Employed Samples</i>	
Parallel Environments	8192
Horizon Steps	50
Update Epochs	20
Mini-batches per Epoch	1
Observation Normalization	True
Reward Normalization	True
Observation History Length	1
<i>Policy Network</i>	
Hidden Layers	$512 \times 256 \times 128$
Activation Function	ELU
Initial Action Std	0.135
Learnable Action Std	True

D. Additional Reward Terms

While Section IV details the main tracking terms, i.e., r_{track} and r_{feet} , here we specify the additional reward terms employed and report the common used coefficients' values (Table V). For other penalization terms, please refer to the `CrispBoosterLocomotionReward` in [2], upon which our implementation is based.

Reward Term	Description	Coefficients
$r_{\text{jnt}}(\mathbf{s}_t, \mathbf{a}_t) = \omega_{\text{jnt}} \exp(-\xi_{\text{jnt}} \ \mathbf{q}_{\text{jnt},t} - \bar{\mathbf{q}}_{\text{jnt}}\ _2^2)$	Tracks upper body joint reference $\bar{\mathbf{q}}_{\text{jnt}}$.	$\omega_{\text{jnt}} = \xi_{\text{jnt}} = 4$
$r_{\text{bh}}(\mathbf{s}_t, \mathbf{a}_t) = -\omega_{\text{bh}} (\frac{w}{b}\mathbf{p}_{t,z} - \frac{w}{b}\bar{\mathbf{p}}_z)^2$	Tracks CoM height reference $\frac{w}{b}\bar{\mathbf{p}}_z$.	$\omega_{\text{bh}} = 10$
$r_{\text{ar}}(\mathbf{s}_t, \mathbf{a}_t) = -\omega_{\text{ar}} \ \mathbf{a}_t - \mathbf{a}_{t-1}\ _2^2$	Penalizes large action changes (smoothness).	$\omega_{\text{ar}} = 3$
$r_{\text{fs}}(\mathbf{s}_t, \mathbf{a}_t) = -\omega_{\text{fs}} \sum_{\square \in \{l,r\}} \ \frac{w}{\square}\dot{\mathbf{p}}_t \cdot \mathbb{C}_{\square}\ _2^2$	Penalizes foot velocity during ground contact (no-slip).	$\omega_{\text{fs}} = 4$

TABLE V: Additional reward terms and their coefficients.

For the sake of completeness, we additionally report (Table VI) the terms we detailed in Section IV.

Reward Term	Description	Coefficients
$r_{\text{feet}}(\mathbf{s}_t, \mathbf{a}_t)$ (See Section IV)	Enforces feet swinging in the allotted period.	$\omega_{\text{feet}} = 6, \delta_{\text{sw}} = 0.1$
$r_{\text{track}}(\mathbf{s}_t, \mathbf{a}_t)$ (See Section IV)	Tracks desired foothold targets.	$\omega_1 = \omega_2 = \omega_3 = 5$ $\xi_1 = \xi_3 = 100, \xi_2 = 200$
$r_{\text{k}}(\mathbf{s}_t, \mathbf{a}_t)$ (See Section IV)	Enforces the knee of the swinging foot to stay above a certain height.	$\omega_{\text{k}} = 4, \xi_{\text{k}} = 200$ $\delta_{\text{k}} = 0.25$

TABLE VI: Tracking and feet swing reward terms and their coefficients.

E. Details on the Employed Velocity-based Policy

In Section V, we compare our approach against a velocity-tracking policy π_{VT} , trained via the *LocoMuJoCo* pipeline [2]. Specifically, π_{VT} shares the same observation vector as our π_{FT} :

$$\mathbf{x}_t = (\mathbf{o}_{\text{proprio},t}; \mathbf{a}_{t-1}; \phi_t; \mathbf{g}_t),$$

where the goal $\mathbf{g}_t$ specifies the target base velocity and height:

$$\mathbf{g}_t = \left({}^w\dot{\bar{\mathbf{p}}}_x; {}^w\dot{\bar{\mathbf{p}}}_y; {}^w\dot{\psi}; {}^w\bar{p}_z \right).$$

Here, ${}^w\dot{\bar{\mathbf{p}}}_x$ and ${}^w\dot{\bar{\mathbf{p}}}_y$ denote the target longitudinal and lateral velocities, ${}^w\dot{\psi}$ is the target angular velocity, and ${}^w\bar{p}_z$ is the target base height. We highlight an additional difference regarding the primary tracking objectives compared to π_{FT} . While the foothold-tracking policy aims at minimizing the error terms detailed in Section IV, π_{VT} utilizes a standard velocity-tracking reward defined as:

$$\begin{aligned} r_{\text{track}}(\mathbf{s}_t, \mathbf{a}_t) = & \omega_1 \exp(-\xi_1 ({}^w\dot{\bar{\mathbf{p}}}_{t,x} - {}^w\dot{\bar{\mathbf{p}}}_x)^2) \\ & + \omega_2 \exp(-\xi_2 ({}^w\dot{\bar{\mathbf{p}}}_{t,y} - {}^w\dot{\bar{\mathbf{p}}}_y)^2) \\ & + \omega_3 \exp(-\xi_3 ({}^w\dot{\psi}_t - {}^w\dot{\psi})^2), \end{aligned}$$

with coefficients set to $\omega_1 = \omega_2 = 1.5$, $\omega_3 = 1$, and $\xi_1 = \xi_2 = \xi_3 = 4$. Base height tracking follows the formulation described in Appendix D, while all other regularization terms remain identical to those used for π_{FT} . Finally, the target velocities are sampled uniformly: ${}^w\dot{\bar{\mathbf{p}}}_x \sim \mathcal{U}(-0.8, 0.8)$ m/s, ${}^w\dot{\bar{\mathbf{p}}}_y \sim \mathcal{U}(-0.5, 0.5)$ m/s, and ${}^w\dot{\psi} \sim \mathcal{U}(-1.5, 1.5)$ rad/s.

F. Training Parameters for the Policy Used for Flat Terrains

The following details hold for both π_{FT} and π_{FT-R} , i.e., the FT policy with targets represented in the stance foot frame and in the root frame, respectively. The policies were trained with PPO for a total of 10^9 timesteps. The table below reports the values used for the goal parameters. Parameters defined by a range are sampled uniformly from the specified interval according to the schedule described in Section IV, while scalar values denote parameters kept constant throughout training.

Parameter	Value / Range
<i>Step Generation</i>	
Movement Direction (θ_{move})	$[-\pi, \pi]$ rad
Feet Direction (θ_{feet})	0 rad
Step Length (d)	$[0.2, 0.5]$ m
θ_{move} Perturbation (α)	$[-2\pi/9, 2\pi/9]$ rad
θ_{feet} Perturbation (β)	$[-\pi/6, \pi/6]$ rad
Height Offset (z)	0 m
<i>Gait Constraints</i>	
Minimum Feet Distance	0.10 m
Hold Still Probability (p_{hold})	0.1
Hold Still Feet Width	0.20 m

For the reward terms, we employed the values detailed in Appendix D, but setting $\omega_2 = 0$ in order to exclude the tracking of the z -component. Moreover, we set $\omega_k = 0$ in order to exclude the swinging knee height term.

G. Training Parameters for the Policy Used for Stairs

In this experiment, the policy was trained using PPO for $1.2 \cdot 10^9$ timesteps, initialized from the flat-terrain policy weights. The table below lists the goal parameters. Ranges imply uniform sampling according to the schedule in Section IV, while scalar values denote constant parameters. The reward terms are detailed in Appendix D.

Parameter	Value / Range
<i>Step Generation</i>	
Movement Direction (θ_{move})	πx rad, $x \sim \mathcal{B}(0.5)$
Feet Direction (θ_{feet})	0 rad
Step Length (d)	$[0.2, 0.4]$ m
θ_{move} Perturbation (α)	$[-\pi/18, \pi/18]$ rad
θ_{feet} Perturbation (β)	$[-\pi/18, \pi/18]$ rad
Height Offset (z)	$[-0.15, 0.15]$ m
<i>Gait Constraints</i>	
Minimum Feet Distance	0.10 m
Hold Still Probability (p_{hold})	0
Hold Still Feet Width	0.20 m

Notably, for this task, we restricted the motion to strictly forward or backward directions, without the possibility of staying still. This is denoted in the table by $\theta_{\text{move}} = \pi x$, where $x \sim \mathcal{B}(0.5)$ represents a Bernoulli variable yielding discrete values $\{0, 1\}$ with equal probability. Notice that the fact that the robot is just trained to go forward or backward it is not restricting the robot's curving capabilities, as demonstrated by the spiral stairs experiment.

On Moving Pillars Terrain. To ensure continuous foothold availability, we employed a dynamic terrain system comprising three dynamic pillars, with a diameter of 0.3 m and friction features matching the floor. These pillars are placed such that whenever a new foot placement target is sampled, the currently free pillar is relocated to the goal position. We define a pillar as free when it is not currently supporting a foot. To ensure valid placements, we employ a rejection sampling mechanism (2048 batched samples) that prevents collisions with the robot's feet or the active stance pillars. Furthermore, to mitigate early termination caused by minor planar tracking errors, we implement an adaptive landing mechanism. During the final portion of the swing phase (specifically when $\phi_t \in [0.25 + \delta_{\text{sw}}, 0.5]$ or $\phi_t \in [0.75 + \delta_{\text{sw}}, 1]$, see Figure 3), the target pillar shifts its xy -position to track the swinging foot's projection. Crucially, the pillar's height z remains fixed to the originally sampled value.

H. High-Level Planners

In this section, we detail the high-level planners employed for both the simulated experiments and the real-world deployment.

Goal-Reaching Planner. For the goal-reaching task, π_{FT} is guided by a deterministic foothold planner. At each gait update, the goal position and base position are expressed in the *stance-foot frame* as ${}^s_g\mathbf{p}$ and ${}^s_b\mathbf{p}$, respectively. If the planar Euclidean distance to the goal falls below a threshold $\|{}^s_g\mathbf{p}_{xy} - {}^s_b\mathbf{p}_{xy}\|_2 \leq \epsilon$, the robot is commanded to stand still, marking task completion. Otherwise, the target for the next swing foot is generated based on the difference vector $\mathbf{d} = {}^s_g\mathbf{p} - {}^s_b\mathbf{p}$. The target yaw is computed as $\theta = \text{clip}(\text{atan2}(\mathbf{d}_y, \mathbf{d}_x), -60^\circ, 60^\circ)$, while the foot target is calculated as ${}^s_{\square}\mathbf{p} = \frac{1}{2}\mathbb{I}_s d_{\text{feet}} \mathbf{v}_{\text{feet}} + d_{\text{max}} \frac{\mathbf{d}}{\|\mathbf{d}\|_2}$, where d_{feet} prescribes the nominal distance between two feet, d_{max} is the maximum distance per step, $\mathbf{v}_{\text{feet}} = [0, 1]^\top$, and $\mathbb{I}_s \in \{-1, 1\}$ is indicator of the swing foot, -1 for right foot swinging and 1 for left foot swinging. Notice that for $\pi_{\text{FT-R}}$ the same procedure applies, but then the policy is fed with the goal $\mathbf{g}_t$ components expressed in the *root frame*. The resulting swing target is clipped to satisfy the feasible constraints to avoid leg crossing. Finally, the planner constructs the goal vector $\mathbf{g}_t$ in the stance-foot frame as described in Section IV: the stance foot reference is set to zero position offset and identity orientation quaternion, while the swing foot reference is the one computed by the planner. Conversely, for the velocity-tracking baseline π_{VT} , a P-controller converts the body-frame planar position error into linear velocity commands (scaled to respect limits), while a separate clipped P-controller generates the yaw rate ω_z to align the heading with the goal. The maximum velocity of the VT baseline is clamped to the trained range comparable to the FT policy. The maximum velocity v_{max} is computed by the maximum step distance d_{max} and gait frequency f_{gait} , i.e., $v_{\text{max}} = d_{\text{max}}/f_{\text{gait}}$. We swept the P-gain to choose the best performing one.

Narrow Bridge Traversal. This task comprises an initial platform, a central narrow bridge, and a final goal platform. While the *approach* and *run-out* sections utilize the previously described goal-reaching planner, the bridge traversal requires a modified planner for lateral walking. The path is defined by a set of ordered waypoints $\mathcal{P} = \{\mathbf{p}_i\}_{i=0}^{N-1}$, tracked via a progress index i . To traverse the bridge, the agent adopts a lateral gait strategy. Consequently, the planner sets the target yaw perpendicular to the local path segment ($\mathbf{p}_{i-1} \rightarrow \mathbf{p}_i$), subject to clipping for meeting yaw constraints. Simultaneously, the foothold position is projected onto the segment connecting $\mathbf{p}_{i-1}$ and $\mathbf{p}_i$, ensuring strict alignment with the bridge centerline.

Cluttered Environment Traversal. In this task, the robot is task with reaching a goal position while avoiding obstacles populating the navigation environment. To generate collision-free foothold targets, we employ a search-based footstep planner [13]. The planner operates on a discretized lattice of stance-foot poses $\mathbf{s} = (x, y, \theta)$, expanding states via alternating support and relative step actions $\mathbf{a} = (\Delta x, \Delta y, \Delta \theta)$. The transition model utilizes a finite action set $\mathcal{A}$ defined by discrete parameters: $\Delta x \in \{0.0, 0.10, 0.20\}$ m, $\Delta y \in \{0.20, 0.25, 0.30, 0.35\}$ m, and $\Delta \theta \in \{0, \pm 10^\circ, \pm 20^\circ, \pm 30^\circ\}$. Given a stance pose $\mathbf{s}$ and an action $\mathbf{a} \in \mathcal{A}$, a successor state $\mathbf{s}' = t(\mathbf{s}, \mathbf{a})$ is generated and snapped to the lattice resolution for hashing. Candidate transitions are filtered by feasibility constraints, including step-length bounds and a minimum inter-foot lateral spacing under alternating support. Furthermore, collision checking discards successors where the foot geometry overlaps with environmental obstacles. Node expansion is prioritized using a Euclidean distance heuristic $h(\mathbf{s}) = \|\mathbf{p}_b - \mathbf{p}_g\|_2$, estimating the cost to the goal $\mathbf{p}_g$. The search algorithm minimizes a weighted evaluation function $f(\mathbf{s}) = g(\mathbf{s}) + w h(\mathbf{s})$, where $g(\mathbf{s})$ represents the accumulated path cost from the start node (with weight $w = 5$). Finally, the computed foothold sequence is encoded into the goal observation $\mathbf{g}_t$ for the policy, as detailed in Section IV.

Stairs Traversal. To evaluate the policy on the stairs climbing task, we implemented a deterministic high-level planner that supplies foothold targets. The obstacle geometry is parameterized by the *step length*, defining the stairs' steps depth; the *step height*, defining the stairs' steps height; the *platform length*, specifying the flat landing separating the ascending and descending sections; and the *feet width*, determining the lateral spacing between the left and right foot trajectories. Additionally, we define a set of *approaching steps* on flat terrain before and after the staircase to stabilize the gait. Based on these dimensions, the planner pre-computes two distinct, parallel sequences of 3D foothold coordinates, $\mathcal{P}_l = \{{}_l^w\mathbf{p}_i\}_{i=0}^{N-1}$ and $\mathcal{P}_r = \{{}_r^w\mathbf{p}_i\}_{i=0}^{N-1}$, for the left and right feet respectively. The sequence follows the fixed topology enforced by the obstacle's terrain profile: an initial flat approach, an ascending staircase, a central landing platform, a descending staircase, and a final flat run-out. During

execution, the robot tracks its progress using two indices, i_l and i_r , pointing to the current target in $\mathcal{P}_l$ and $\mathcal{P}_r$. The starting swing foot is selected based on the robot's initial yaw orientation. At each gait phase switch (swing $\rightarrow$ stance), the indices are updated to enforce an alternating gait: if the right foot is becoming the swing foot, its target index advances as $i_r \leftarrow i_l + 1$; conversely, if the left foot swings, $i_l \leftarrow i_r + 1$. Finally, the planner outputs the goal g_t in the *stance-foot frame*, as described in Section IV. Specifically, the reference for the incoming stance foot is set to a zero position and identity orientation offset, while the reference for the incoming swing foot is computed as the relative transformation between the current stance foot and the target foothold retrieved from the plan.

Real-World Vision-Based Planner. As mentioned in Section VI, the hardware experiments on the Booster T1 humanoid rely on a high-level planner driven by a computer vision module exploiting the onboard camera. The task involves navigating a corridor by stepping onto manually placed circular markers. We employ the onboard color camera, a RealSense D455, to detect these targets. Specifically, at each gait switch instant, the planner fuses RGB detection with depth measurements to extract the next foothold position in the camera frame. Using the robot's kinematics, this target is transformed into the current stance-foot frame to compute the relative position offsets required by the policy as detailed in Section IV (keeping the stance foot offsets at zero). Orientation targets are determined heuristically to ensure kinematic feasibility. Finally, as already described in Section VI, to account for real-world latencies and imperfect ground contacts, we do not adhere strictly to the open-loop gait clock. Instead, the gait signal is frozen (repeating the last value) until the relative vertical distance between the stance and swing feet falls below a safety threshold, ensuring robust ground contact before transitioning.

I. Section V's Extended Tables

Here, we report extended tables for the experiments conducted in Section V. Specifically, we include results for the straight and spiral staircase traversal experiments.

Step Length (m)	Step Height (m)									
	0.01	0.02	0.04	0.08	0.10	0.11	0.115	0.12	0.125	0.13
0.300	100%	100%	100%	95%	76%	60%	52%	56%	38%	12%
	R:0 B:0 0.033 $\pm$ 0.006	R:0 B:0 0.034 $\pm$ 0.003	R:0 B:0 0.036 $\pm$ 0.005	R:3 B:2 0.059 $\pm$ 0.089	R:19 B:5 0.071 $\pm$ 0.049	R:40 B:0 0.081 $\pm$ 0.048	R:34 B:14 0.090 $\pm$ 0.048	R:39 B:5 0.083 $\pm$ 0.041	R:47 B:15 0.095 $\pm$ 0.038	R:70 B:18 0.108 $\pm$ 0.039
0.325	100%	100%	100%	100%	100%	92%	89%	74%	30%	6%
	R:0 B:0 0.034 $\pm$ 0.004	R:0 B:0 0.034 $\pm$ 0.005	R:0 B:0 0.036 $\pm$ 0.003	R:0 B:0 0.038 $\pm$ 0.005	R:0 B:0 0.041 $\pm$ 0.006	R:5 B:3 0.050 $\pm$ 0.029	R:11 B:0 0.053 $\pm$ 0.027	R:22 B:4 0.068 $\pm$ 0.042	R:54 B:16 0.104 $\pm$ 0.045	R:70 B:14 0.115 $\pm$ 0.038
0.350	100%	100%	100%	100%	99%	100%	96%	89%	47%	19%
	R:0 B:0 0.039 $\pm$ 0.010	R:0 B:0 0.038 $\pm$ 0.011	R:0 B:0 0.041 $\pm$ 0.018	R:0 B:0 0.040 $\pm$ 0.005	R:1 B:0 0.045 $\pm$ 0.022	R:0 B:0 0.045 $\pm$ 0.016	R:2 B:2 0.050 $\pm$ 0.022	R:7 B:4 0.066 $\pm$ 0.056	R:34 B:19 0.106 $\pm$ 0.060	R:58 B:23 0.136 $\pm$ 0.057
0.375	100%	100%	100%	100%	100%	99%	86%	88%	87%	46%
	R:0 B:0 0.049 $\pm$ 0.012	R:0 B:0 0.052 $\pm$ 0.022	R:0 B:0 0.048 $\pm$ 0.010	R:0 B:0 0.048 $\pm$ 0.011	R:0 B:0 0.052 $\pm$ 0.010	R:0 B:1 0.054 $\pm$ 0.017	R:4 B:10 0.058 $\pm$ 0.019	R:8 B:4 0.079 $\pm$ 0.100	R:6 B:7 0.087 $\pm$ 0.112	R:23 B:31 0.113 $\pm$ 0.086

TABLE VII: **Traversing Straight Stairs.** Each cell reports success rate, riser-contact falls (R) and balance falls (B), and mean $\pm$ std 3D foot placement error (m). 100 evaluations with initial robot yaw sampled from $\mathcal{U}(-\pi/6, \pi, 6)$.

Step Length (m)	Step Height (m)									
	0.01	0.02	0.04	0.08	0.10	0.11	0.115	0.12	0.125	0.13
0.300	100%	99%	96%	97%	82%	40%	21%	14%	19%	14%
	R:0 B:0 0.057 $\pm$ 0.008	R:0 B:1 0.053 $\pm$ 0.012	R:4 B:0 0.049 $\pm$ 0.020	R:2 B:1 0.045 $\pm$ 0.017	R:17 B:1 0.061 $\pm$ 0.029	R:40 B:20 0.082 $\pm$ 0.028	R:61 B:18 0.104 $\pm$ 0.031	R:71 B:15 0.114 $\pm$ 0.034	R:73 B:8 0.112 $\pm$ 0.030	R:72 B:14 0.132 $\pm$ 0.053
0.325	98%	97%	97%	97%	91%	77%	58%	66%	29%	17%
	R:0 B:2 0.066 $\pm$ 0.015	R:2 B:1 0.066 $\pm$ 0.020	R:3 B:0 0.061 $\pm$ 0.020	R:2 B:1 0.049 $\pm$ 0.044	R:3 B:6 0.060 $\pm$ 0.025	R:15 B:8 0.066 $\pm$ 0.035	R:20 B:22 0.085 $\pm$ 0.040	R:31 B:3 0.082 $\pm$ 0.050	R:59 B:12 0.097 $\pm$ 0.038	R:62 B:21 0.104 $\pm$ 0.031
0.350	94%	97%	97%	96%	94%	91%	84%	79%	57%	28%
	R:2 B:4 0.079 $\pm$ 0.039	R:1 B:2 0.082 $\pm$ 0.030	R:1 B:2 0.080 $\pm$ 0.029	R:0 B:4 0.061 $\pm$ 0.068	R:4 B:2 0.065 $\pm$ 0.088	R:3 B:6 0.074 $\pm$ 0.098	R:8 B:8 0.084 $\pm$ 0.095	R:13 B:8 0.093 $\pm$ 0.107	R:25 B:18 0.097 $\pm$ 0.059	R:55 B:17 0.132 $\pm$ 0.061
0.375	96%	97%	98%	91%	90%	96%	93%	74%	54%	17%
	R:1 B:3 0.092 $\pm$ 0.044	R:2 B:1 0.095 $\pm$ 0.051	R:1 B:1 0.099 $\pm$ 0.026	R:4 B:5 0.081 $\pm$ 0.038	R:4 B:6 0.090 $\pm$ 0.137	R:1 B:3 0.069 $\pm$ 0.048	R:1 B:6 0.089 $\pm$ 0.126	R:7 B:19 0.097 $\pm$ 0.067	R:16 B:30 0.125 $\pm$ 0.107	R:45 B:37 0.155 $\pm$ 0.097

TABLE VIII: **Traversing Spiral Stairs.** Each cell reports success rate, riser-contact falls (R) and balance falls (B), and mean $\pm$ std 3D foot placement error (m). 100 evaluations with initial robot yaw sampled from $\mathcal{U}(-\pi/6, \pi, 6)$.